Vehicle Scheduling System

Visual Reference

Diagrams for the parts that text alone can't carry.

Domain Model · Use Cases · Architecture · Workflows · Conflict Types

Use Case Map

Each page → its API operations → domain logic → the entities it touches

PAGE

Schedule List

`/schedule`

Browse · Create · Delete · Auto-generate

API OPERATIONS

- `GET /services`
- `GET /services/{id}`
- `POST /services`
- `DELETE /services/{id}`
- `POST /schedules/generate`

DOMAIN LOGIC

- Aggregate read (list + expanded detail)
- Cascade delete
- Auto-gen runs BFS + timetable + 6 conflict checks

ENTITIES TOUCHED

Service · Vehicle · Block

PAGE

Schedule Editor

`/schedule/:id/edit`

Edit one service's route

API OPERATIONS

- `GET /services/{id}`
- `PATCH /services/{id}/route`

DOMAIN LOGIC

- BFS connectivity check
- Compute timetable (dwell + traversal)
- Detect 6 conflict types → 409 on failure

ENTITIES TOUCHED

Service · Block · NodeConnection

PAGE

Config

`/config`

Block table + Vehicle list

API OPERATIONS

- `GET /blocks`
- `PATCH /blocks/{id}`
- `GET /vehicles`

DOMAIN LOGIC

- Update traversal_time_seconds
- (existing services keep their timetables until re-saved)

ENTITIES TOUCHED

Block · Vehicle

Service Lifecycle

From creation to a validated, persisted route

1. **POST /api/services**
   Create with name + vehicle_id → empty route
2. **PATCH /api/services/{id}/route**
   Send stops, dwell times, start_time
3. **Validate connectivity (BFS)**
   Walk directed block graph between consecutive stops
4. **Compute timetable**
   Arrival / departure per node from dwell + traversal
5. **Conflict detection**
   Sweep against all other services in memory

200 OK · Persist

Service stored with route + timetable.
Conflict detection passed.

409 Conflict

Returned with structured details:

- vehicle overlap
- vehicle discontinuity
- block occupancy
- interlocking
- low battery
- insufficient charge

Request was valid in isolation; conflict arose from current state.

Route Building & Timetable

Users pick stops; the system fills the rest

1 User Input

```
stops = [
    P1A  (dwell 60s),
    P2A  (dwell 45s),
]
start_time = T
```

→

2 BFS Inference

Walks the directed block adjacency graph, shortest chain between consecutive stops.

Most stop pairs have exactly one valid path.

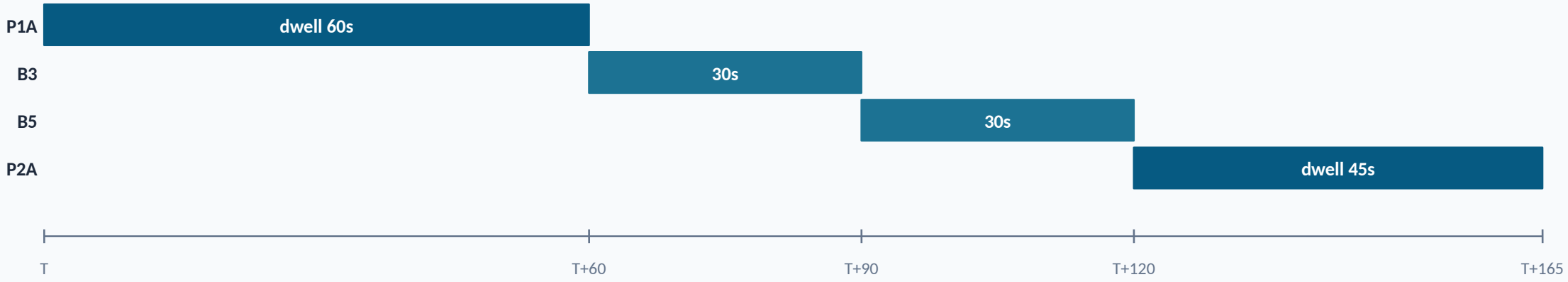
→

3 Full Route + Timetable

P1A → B3 → B5 → P2A

Arrival/departure computed per node from dwell + traversal times.

Timeline (example): T = start_time



Conflict Detection — 6 Types

Returned in the 409 response when a route is saved

Vehicle Overlap

Same vehicle in two services with overlapping time windows.

Vehicle Discontinuity

Consecutive services on same vehicle don't connect spatially.

Block Occupancy

Two services occupy the same block at overlapping times.

Interlocking

Blocks in the same interlocking group occupied simultaneously.

Low Battery

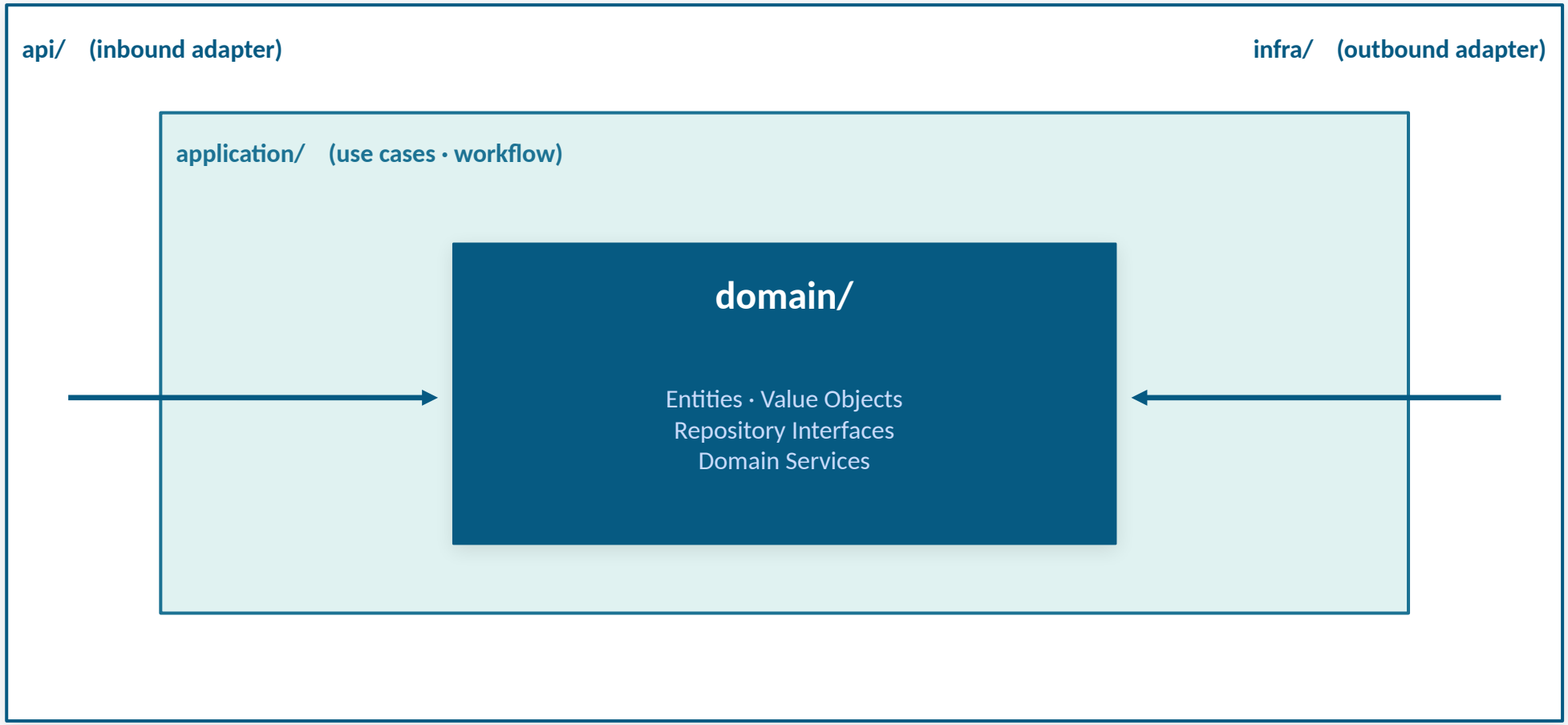
Battery drops below 30 outside the yard.

Insufficient Charge

Vehicle departs the yard with battery below 80.

Hexagonal Architecture

Ports & adapters — dependencies always point inward



Nothing in domain/ imports anything outside it. Enforced by import-linter at CI time.

Domain Model — Aggregates & Entities

DDD building blocks and how they relate

Station (aggregate root)

- id (UUID)
- name
- is_yard : bool
- platforms[]

1 *

Platform (entity)

- id (UUID)
- name

Vehicle (aggregate root)

- id (UUID)
- name
- battery

1 *

Service (aggregate root)

- id, name, vehicle_id
- route : Node[] (ordered: Platform | Block | Yard)
- timetable : TimetableEntry[] (arr / dep per node)

Block (aggregate root)

- id, name
- group (interlocking)
- traversal_time_seconds

NodeConnection (shared kernel)

- from_id → to_id (directed graph edge)
- BFS over this graph fills the route between stops

Service.route and Service.timetable are stored as JSONB — loaded and saved as a whole.

Domain Model — Value Objects & Read Models

Immutable types embedded in aggregates, plus query-side projections

Node (value object)

- id (UUID)
- type : NodeType (BLOCK | PLATFORM | YARD)

TimetableEntry (value object)

- order
- node_id (UUID)
- arrival : EpochSeconds
- departure : EpochSeconds

LayoutData (read model)

- positions : dict[UUID, tuple[float, float]]
- junction_blocks : dict[tuple[UUID, UUID], UUID]